Chapter 7 JavaScript: Control Statements, Part 1

Internet & World Wide Web
How to Program, 5/e

ftp://111webdesign:webdesign@120.114.190.22:4000

＊培養問題解決能力

Outline

- 7.1 Introduction
- 7.2 Algorithms
- 7.3 Pseudocode
- 7.4 Control Statements
- 7.5 if Selection Statement
- 7.6 if…else Selection Statement
- **7.7** while Repetition Statement
- **7.8** Formulating Algorithms: CounterControlled Repetition
- **7.9** Formulating Algorithms: SentinelControlled Repetition
- **7.10** Formulating Algorithms: Nested Control Statements
- **7.11** Assignment Operators
- **7.12** Increment and Decrement Operators
- **7.13** Web Resources

7.7 while Repetition Statement

- While 迴圈
  - Allows you to specify that an action is to be repeated while some condition remains true
  - Until the condition becomes false and repetition terminates
    - 註：當符合某條件時，重覆(迴圈)執行某個action或命令，直到條件不成立則停止
  - The body of a loop may be a single statement or a block
    - 註：迴圈執行內容可以是單一陳述(...;)或執行區塊/多陳述({...})

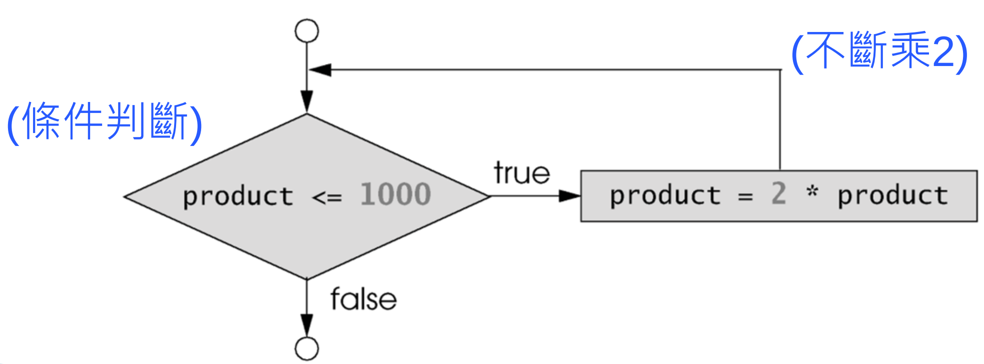


Fig. 7.5 | Flowcharting the while repetition statement.

7.8 Formulating Algorithms: Counter-Controlled Repetition

- Counter-controlled repetition 計數控制的重復 (迴圈)
  - Also called definite repetition 明確的重復 (迴圈), because the number of repetitions is known (已知重覆次數) before the loop begins
  - Example：
    - A *total* is a variable in which a script accumulates the sum of a series of values. Initially, should be initialized to zero.

      註：total表示加總值, 初始時為0
    - A *counter* is a variable a script uses to count—typically in a repetition statement

      註：*counter*用於計數 (次數: 初始化為1)

Pseudocode (計算：班上10位同學的平均成績)

```
Set total to zero                                     //總合變數，初始為0
Set grade counter to one                              //計數器，初始為1

While grade counter is less than or equal to ten      //迴圈條件檢查
    Input the next grade
    Add the grade into the total                      //計算：累記分數
    Add one to the grade counter                      //計算：計數器加1

Set the class average to the total divided by ten     //計算：平均分數
Print the class average
```

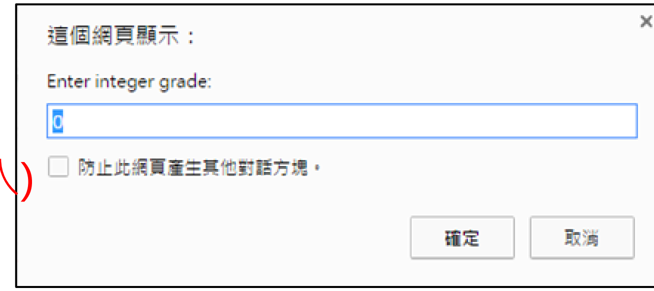
Fig. 7.6 | Pseudocode algorithm that uses counter-controlled repetition to solve the class-average problem.

(Trace 程式碼)

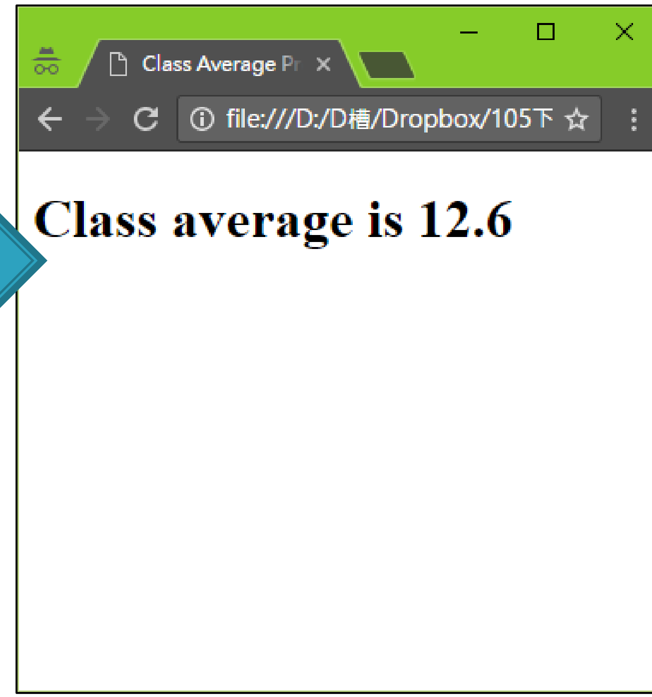
1) average.html

```
<!DOCTYPE html>

<!-- Fig. 7.7: average.html -->
<!-- Counter-controlled repetition to calculate a class average. -
<html>
   <head>
      <meta charset = "utf-8">
      <title>Class Average Program</title>
      <script>
                                                        (初始化)
         var total; // sum of grades
         var gradeCounter; // number of grades entered
         var grade; // grade typed by user (as a string)
         var gradeValue; // grade value (converted to intetger)
         var average; // average of all grades

         // Initialization Phase
         total = 0; // clear total
         gradeCounter = 1; // prepare to loop

         // Processing Phase                          (while迴圈)
         while ( gradeCounter <= 10 ) // loop 10 times
         {

            // prompt for input and read grade from user
            grade = window.prompt( "Enter integer grade:", "0" );
                       //使用對話框進行輸入成績
            // convert grade from a string to an integer
            gradeValue = parseInt( grade ); //字串轉成整數

            // add gradeValue to total
            total = total + gradeValue;

            // add 1 to gradeCounter
            gradeCounter = gradeCounter + 1;
         } // end while

         // Termination Phase
         average = total / 10;   // calculate the average

         // display average of exam grades
         document.writeln(
            "<h1>Class average is " + average + "</h1>" );

      </script>
   </head><body></body>
</html>
```

(連續輸入10筆成績)



(動態寫入)



7.8 Formulating Algorithms: Counter-Controlled Repetition

- JavaScript represents all numbers as floating-point numbers in memory
  - 註：javascript在記憶體是使用「浮點數」代表數字
- The computer allocates only a fixed amount of space to hold such a value, so the stored floating-point value can only be an approximation
  - 註：電腦只配置固定空間儲存數值(小數部分只會是近似值)

- Sentinel-controlled repetition 哨兵(或稱旗標flag)控制重復的迴圈
  - Special value called a sentinel value (also called a signal value 訊號值, a dummy value 虛擬值 or a flag value 旗標值) indicates the end of data entry
    - 註：使用特定的值，表示迴圈結束(ex: “-1”)
  - Often is called indefinite repetition (不明確的重復), because the number of repetitions is not known in advance (未知何時結束)
  - Choose a sentinel value that cannot be confused with an acceptable input value
    - 註：旗標值挑選，不可和計算值混淆 (例如：成績0~100；旗標可挑-1，不可挑0~100，否則造成混淆)

average2.html

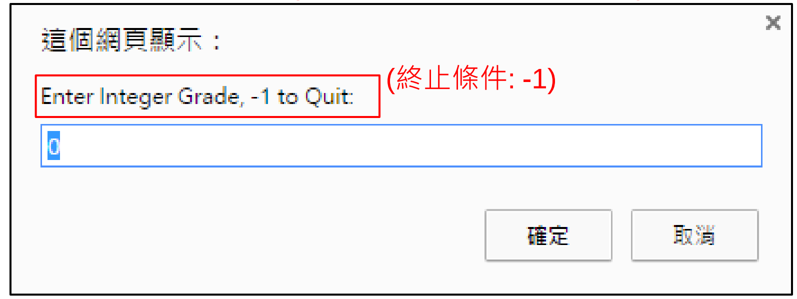
```
<!-- Sentinel-controlled repetition to calculate a class average.
<html>
   <head>
      <meta charset = "utf-8">
      <title>Class Average Program: Sentinel-controlled Repetition
      <script>

         var total; // sum of grades
         var gradeCounter; // number of grades entered
         var grade; // grade typed by user (as a string)
         var gradeValue; // grade value (converted to integer)
         var average; // average of all grades


         // Initialization phase
         total = 0; // clear total
         gradeCounter = 0; // prepare to loop

         // Processing phase
         // prompt for input and read grade from user
         grade = window.prompt(
              "Enter Integer Grade, -1 to Quit:", "0" );
         // convert grade from a String to an integer
         gradeValue = parseInt( grade );

         while ( gradeValue != -1 )
         {
            // add gradeValue to total
            total = total + gradeValue;
            // add 1 to gradeCounter
            gradeCounter = gradeCounter + 1;
            // prompt for input and read grade from user
            grade = window.prompt(
                 "Enter Integer Grade, -1 to Quit:", "0" );
            // convert grade from a String to an integer
            gradeValue = parseInt( grade );
         } // end while
         // Termination phase
         if ( gradeCounter != 0 )
         {
            average = total / gradeCounter;

            // display average of exam grades
            document.writeln(
               "<h1>Class average is " + average + "</h1>" );
         } // end if
         else
            document.writeln( "<p>No grades were entered</p>" );

      </script>
   </head><body></body>
</html>
```

(初始化)

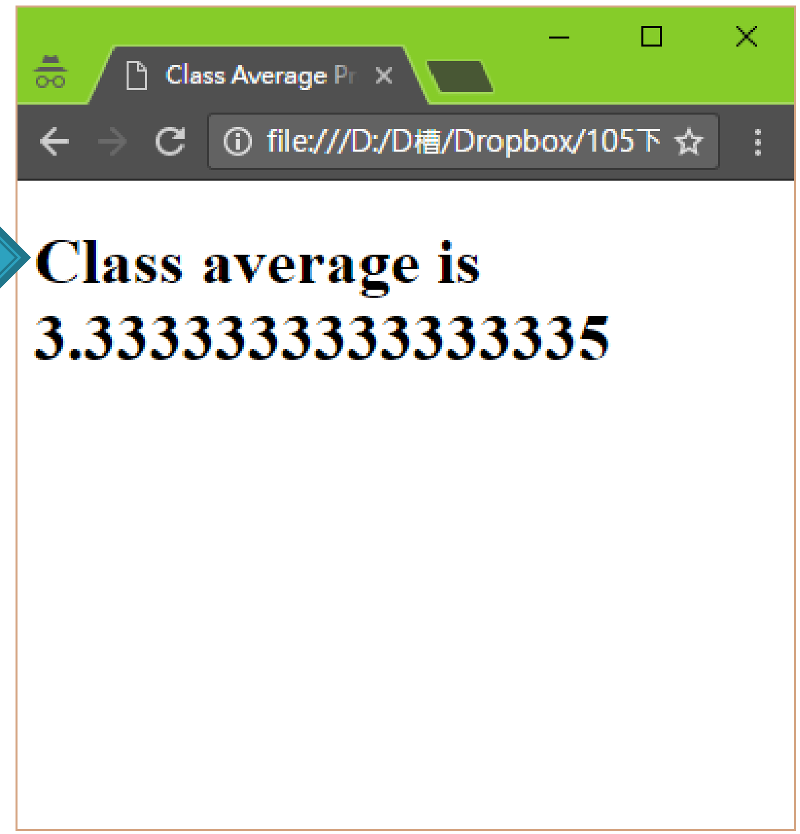
//判斷條件 != -1

//避免平均除以0

(連續輸入10筆成績，-1離開)



(終止條件: -1)



In-Class Exercise #1

- Write a JavaScript program that reads in an arbitrary number of positive integers, and outputs the largest and the smallest integers.
- **課中練習**：請使用JavaScript程式，讀取網頁使用者輸入的任意個正整數 (請設定終止條件，如：-1)最後在網頁上顯示所輸入的：1) 最大值、2) 最小值及3) 平均值。

(最後留時間給同學們練習)

7.10 Formulating Algorithms: **Nested** Control Statements

巢狀控制陳述 (多層控制敘述，如: **while** 裡加入**if else**)

- Control structures may be nested (巢狀) inside of one another

3) analysis.html

```
<!DOCTYPE html>

<!-- Fig. 7.11: analysis.html -->
<!-- Examination-results calculation. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Analysis of Examination Results</title>
      <script>

         // initializing variables in declarations
         var passes = 0; // number of passes
         var failures = 0; // number of failures
         var student = 1; // student counter
         var result; // an exam result

         // process 10 students; counter-controlled loop
         while ( student <= 10 )
         {
            result = window.prompt( "Enter result (1=pass,2=fail)", "0" );

            if ( result == "1" )
               passes = passes + 1;
            else
               failures = failures + 1;

            student = student + 1;
         } // end while

         // termination phase
         document.writeln( "<h1>Examination Results</h1>" );
         document.writeln( "<p>Passed: " + passes +
            "; Failed: " + failures + "</p>" );

         if ( passes > 8 )
            document.writeln( "<p>Bonus to instructor!</p>" );

      </script>
   </head><body></body>
</html>
```

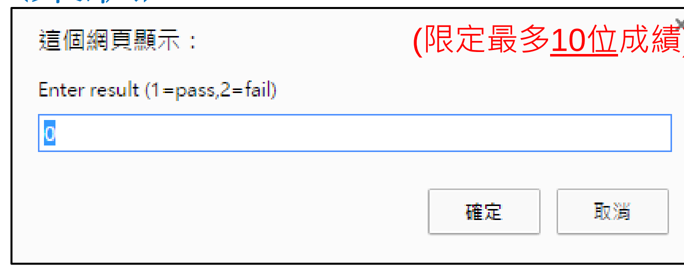
//統計班上: pass(“1”) 或 failures(“2”)人數，共10位

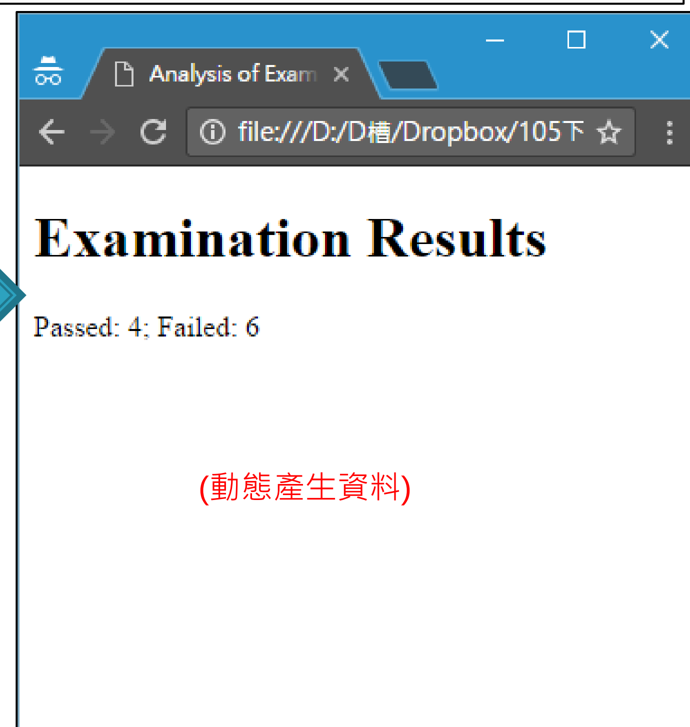
(巢狀)

累計pass

累計failure

(限定最多10位成績)





(動態產生資料)

7.11 Assignment Operators

簡化計算方式

(配置 賦值)

- JavaScript provides the arithmetic assignment operators += 加法, -= 減法, *= 乘法, /= 除法 and %= 取餘數, which abbreviate (簡化/縮寫) certain expressions.

(同一變數，做運算)

Assignment operator	Initial value of variable	Sample expression	Explanation	Assigns
+= (加法)	c = 3	c $\boxed{+=}$ 7	c = c + 7	10 to c
-= (減法)	d = 5	d $\boxed{-=}$ 4	d = d - 4	1 to d
*= (乘法)	e = 4	e $\boxed{*=}$ 5	e = e * 5	20 to e
/= (除法)	f = 6	f $\boxed{/=}$ 3	f = f / 3	2 to f
%= (取餘數)	g = 12	g $\boxed{\%=}$ 9	g = g % 9	3 to g

Fig. 7.12 | Arithmetic assignment operators.

7.12 Increment and Decrement Operators

【註】：
- ++a; //當行+1
- a++; //下一行+1

簡化："變數加1、減1"的方式

- The increment operator, ++, and the decrement operator, --, increment or decrement a variable by 1, respectively.

加減號在變數前(ex: ++a; --a;)

  - If the operator is prefixed (加於字首) to the variable, the variable is incremented or decremented by 1, then used in its expression.
    - 補充：當行先運算 (加1、減1)

加減號在變數後(ex: a++; a--;)

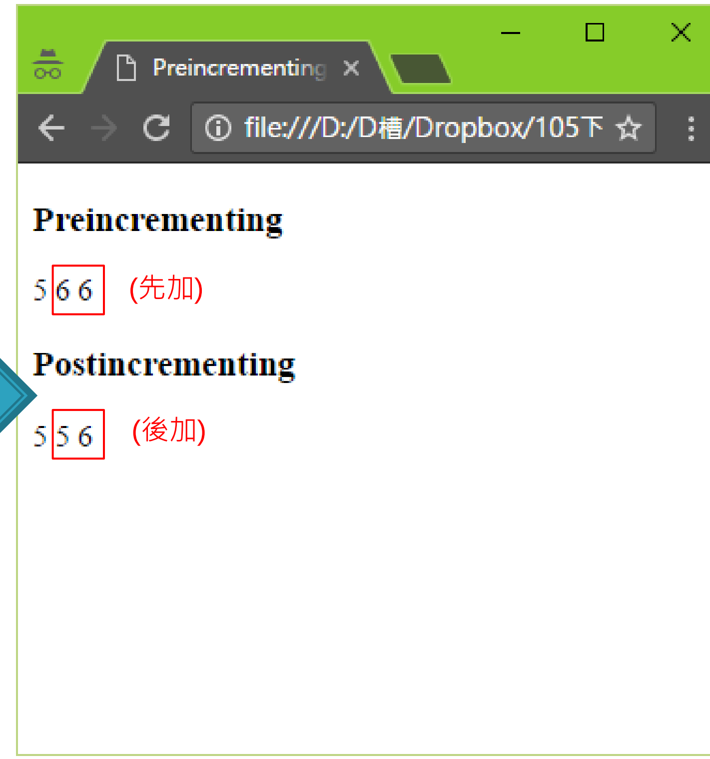
  - If the operator is postfixed (加於字尾) to the variable, the variable is used in its expression, then incremented or decremented by 1.
    - 補充：隔行才運算 (加1、減1)

Operator	Example	Called	Explanation
++	++a (先加)	preincrement	Increment a by 1, then use the new value of a in the expression in which a resides.
++	a++ (後加)	postincrement	Use the current value of a in the expression in which a resides, then increment a by 1.
--	--b	predecrement	Decrement b by 1, then use the new value of b in the expression in which b resides.
--	b--	postdecrement	Use the current value of b in the expression in which b resides, then decrement b by 1.

7.12 Increment and Decrement Operators

4) increment.html

```
<!DOCTYPE html>

<!-- Fig. 7.14: increment.html -->
<!-- Preincrementing and Postincrementing. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Preincrementing and Postincrementing</title>
      <script>
         var c;
         c = 5; //原始值為5
         document.writeln( "<h3>Preincrementing</h3>" );
         document.writeln( "<p>" + c ); // prints 5
         // increments then prints 6
(先加)   document.writeln( " " + ++c );//當行+1
         document.writeln( " " + c + "</p>" ); // prints 6

         c = 5;
         document.writeln( "<h3>Postincrementing</h3>" );
         document.writeln( "<p>" + c ); // prints 5
         // prints 5 then increments
(後加)   document.writeln( " " + c++ );//隔行才+1
         document.writeln( " " + c + "</p>" ); // prints 6

      </script>
   </head><body></body>
</html>
```



7.12 Summary

(運算符號之結合性)

Operator	Associativity	Type
++ -- (ex: x++;)	right to left	unary (一元運算)
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== != === !=== (嚴格型態判斷)	left to right	equality
?:	right to left	conditional
= += -= *= /= %= (ex: x +=5;)	right to left (右邊算完，才給左邊)	assignment

Fig. 7.15 | Precedence and associativity of the operators discussed so far.

Chapter 8 JavaScript: Control Statements, Part 2 (延續)

Internet & World Wide Web How to Program, 5/e

ftp://111webdesign:webdesign@120.114.190.22:4000

＊互相學習，教學相長

OBJECTIVES

In this chapter you'll:

- Learn the essentials of counter-controlled repetition
- Use the `for` and `do`…`while` repetition statements to execute statements in a program repeatedly.
- Perform multiple selection using the `switch` selection statement.
- Use the `break` and `continue` program-control statements
- Use the logical operators to make decisions.

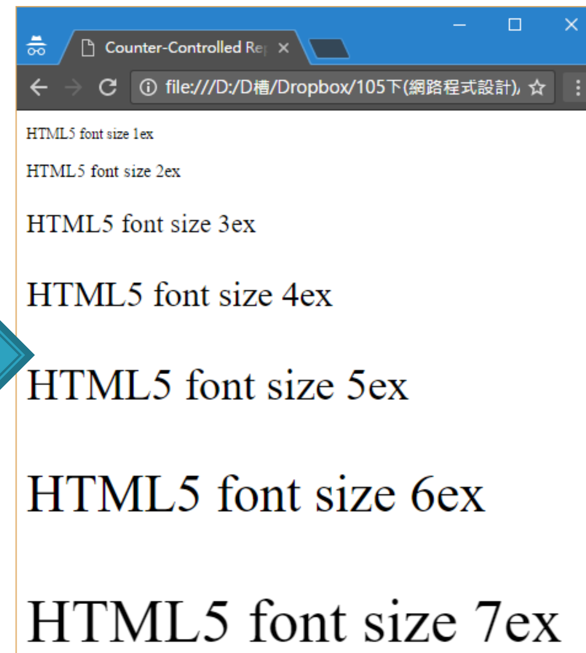
8.2 Counter-Controlled Repetition

計數控制重復的迴圈特性

- Counter-controlled repetition requires (需定義明確)
  (4 項元素)
  1. name of a control variable (控制變數名稱)
  2. initial value of the control variable (初始值)
  3. the increment 遞增值 (or decrement 遞減值) by which the control variable is modified each time through the loop
  4. the condition (條件判斷是否成立、並繼續) that tests for the final value of the control variable to determine whether looping should continue

1) WhileCounter.html

```
<!DOCTYPE html>

<!-- Fig. 8.1: WhileCounter.html -->
<!-- Counter-controlled repetition. -->
<html>
   <head>
      <meta charset=  "utf-8">
      <title>Counter-Controlled Repetition</title>
      <script>
            //變數名稱 //初始化
         var counter = 1; // initialization

         while ( counter <= 7) // repetition condition
         {         //判斷條件
            document.writeln( "<p style = 'font-size: " +
               counter + "ex'>HTML5 font size " + counter + "ex</p>" );
            ++counter; // increment //遞增1
         } //end while

      </script>
   </head><body></body>
</html>
```



(動態控制網頁呈現)

8.3 for Repetition Statement

計數控制重復的迴圈(第2種)： (跟c語言類似)

for(...初始化...; ...條件判斷...; ...遞增/遞減...){...動作命令...}

for statement (for迴圈)
- Counter-controlled repetition with a control variable (使用控制變數)
- Can use a block {…} to put multiple statements into the body

- for statement header (for迴圈的敘述) contains three factors

(3 項重要元素)
1. Initialization (初始化)
2. Condition (條件判斷)
3. Increment Expression (遞增/遞減式)
   - 註：就像在body結束前的陳述

- 註：通常初始化、遞增/遞減式只放入一個陳述
- 註：若用 < 取代 <= 條件判斷，可能會有差1次的情況。

2) ForCounter.html

```
<!DOCTYPE html>

<!-- Fig. 8.2: ForCounter.html -->
<!-- Counter-controlled repetition with the for statement. -->
<html>
   <head>
      <meta charset="utf-8">
      <title>Counter-Controlled Repetition</title>
      <script>

         // Initialization, repetition condition and
         // incrementing are all included in the for
         // statement header.
         for ( var counter = 1; counter <= 7; ++counter )
            document.writeln( "<p style = 'font-size: " +
               counter + "ex'>HTML5 font size " + counter + "ex</p>" );

      </script>
   </head><body></body>
</html>
```

(控制變數、初始化 (條件判斷) (遞增式)

8.3 for Repetition Statement (Cont.)

- The three expressions in the for statement are optional
  - 註：以上三敘述是選擇性使用(可不填, for (i=1 ; ;){…})
- The two semicolons (分號“;”) in the for statement are required
  - 註：分號”;”為**必填**，否則語法錯誤
- The initialization (初始化), loop condition (迴圈條件) and increment expressions (遞增式) of a for statement can contain arithmetic expressions
  - 註：初始式、條件式、遞增式，可包含複雜計算 (ex: for (i = x +5; i-3<2; i = i + x){…})
- The “increment” of a for statement may be negative (負值), in which case it is called a decrement (遞減) and the loop actually counts downward (下降)
  - 註：遞增變量為負值，則為遞減
- If the loop condition initially is false, the body of the for statement is not performed
  - 註：若迴圈條件初始便不成立，則不進入迴圈 (成立，才會進去執行)

8.3 for Repetition Statement (flowchart)

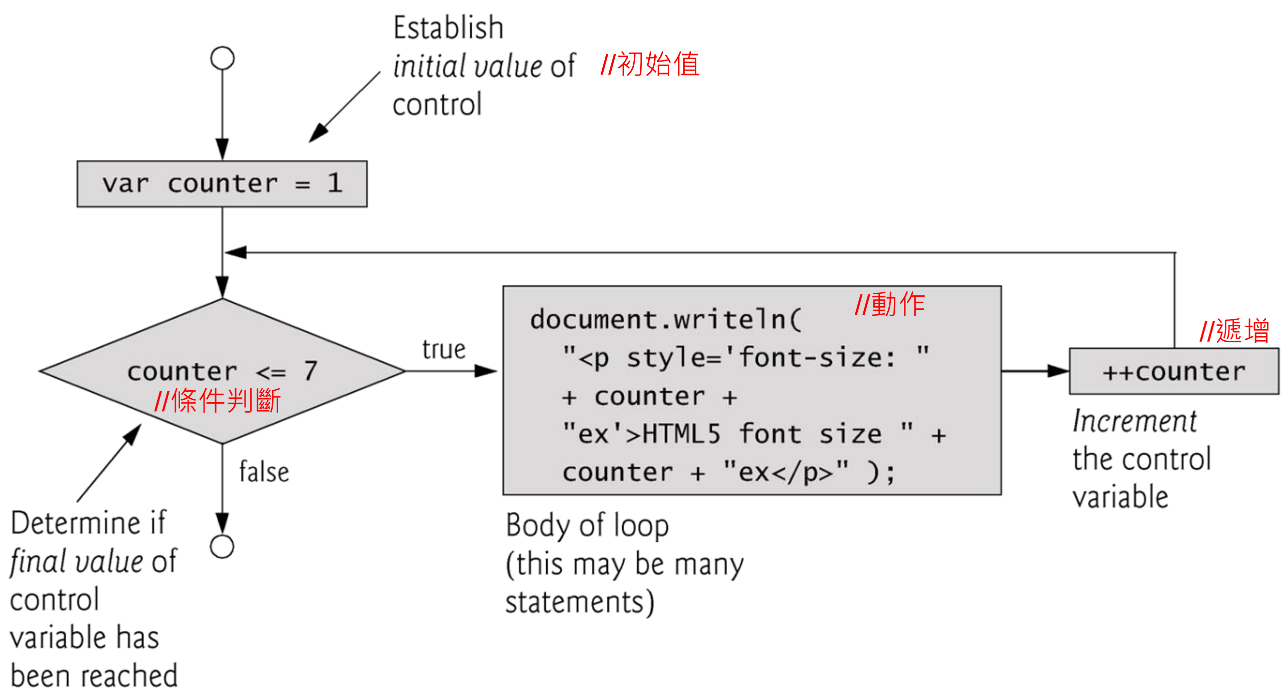


Fig. 8.4 | for repetition statement flowchart.

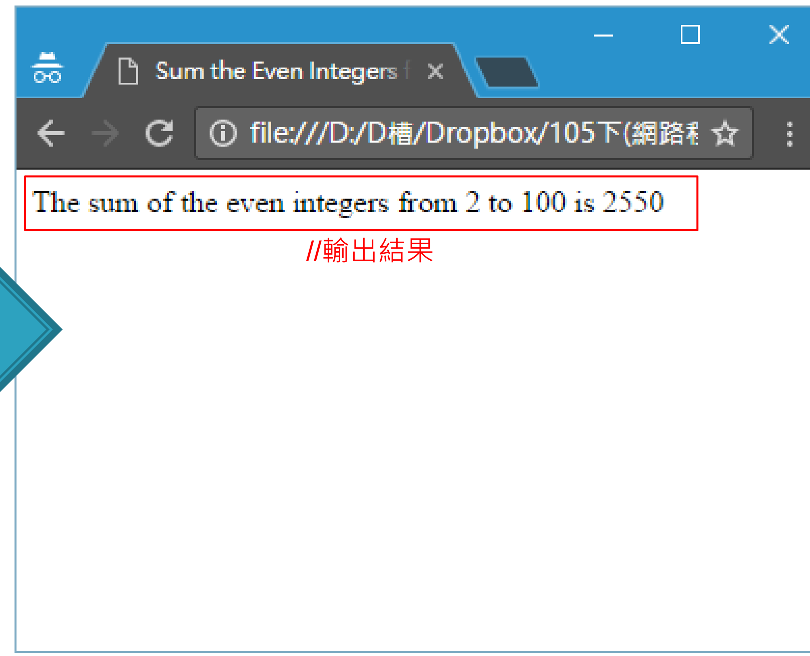
8.4 Examples Using the for Statement

- Example: To sum the even integers (偶數) from 2 to 100.

3) Sum.html

```
<!DOCTYPE html>

<!-- Fig. 8.5: Sum.html -->
<!-- Summation with the for repetition structure. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Sum the Even Integers from 2 to 100</title>
      <script>

         var sum = 0;

         for ( var number = 2; number <= 100; number += 2)
            sum += number;

         document.writeln( "The sum of the even integers " +
            "from 2 to 100 is " + sum );

      </script>
   </head><body></body>
</html>
```

// for 迴圈



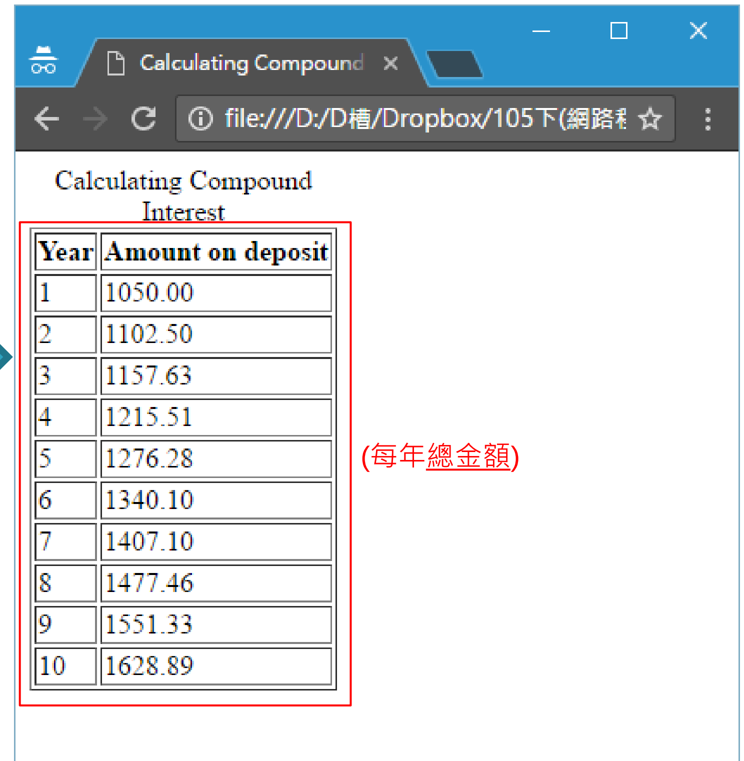
//輸出結果

8.4 Examples Using the for Statement

- Example 2: 計算複利，本金1000元，利率5%，為期10年.
  - 註：JavaScript沒有指數operator，使用Math物件：Math.pow(x, y); //計算x^y.

4) Interest.html

```
<!DOCTYPE html>

<!-- Fig. 8.6: Interest.html -->
<!-- Compound interest calculation with a for loop. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Calculating Compound Interest</title>

      <script>
         var amount; // current amount of money
         var principal = 1000.00; // principal amount //本金
         var rate = 0.05; // interest rate  //利率

         document.writeln( "<table border =1>" ); // begin th
         document.writeln(
            "<caption>Calculating Compound Interest</caption>
         document.writeln(
            "<thead><tr><th>Year</th>" ); // year column heading
         document.writeln(
            "<th>Amount on deposit</th>" ); // amount column heading
         document.writeln( "</tr></thead><tbody>" );
                                                   // 複利 = 本金*(1+利率)年
         // output a table row for each year
         for ( var year = 1; year <= 10; ++year )
         {
            amount = principal * Math.pow( 1.0 + rate, year );

            document.writeln( "<tr><td>" + year +
                 "</td><td>" + amount.toFixed(2) + "</td></tr>" );
         } //end for

         document.writeln( "</tbody></table>" );

      </script>
   </head><body></body>
</html>
```



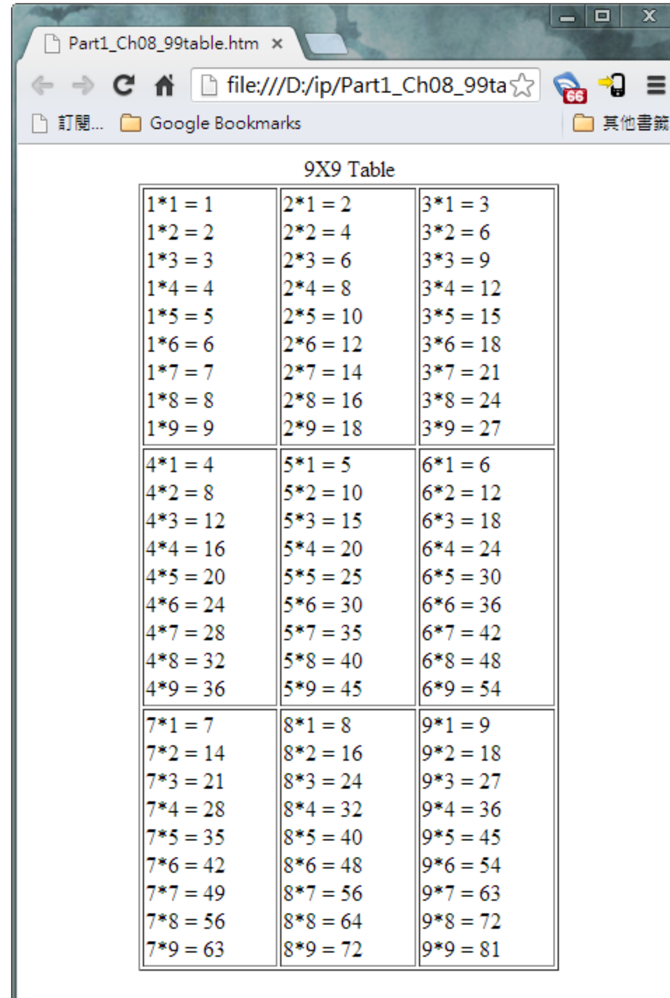
Calculating Compound Interest

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.63
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

(每年總金額)

In-Class Exercise #2

- Write a JavaScript program that produces the following 9X9 table.
- 課中練習：請用JavaScript建立如右圖的九九乘法表。



9X9 Table

1*1 = 1 1*2 = 2 1*3 = 3 1*4 = 4 1*5 = 5 1*6 = 6 1*7 = 7 1*8 = 8 1*9 = 9	2*1 = 2 2*2 = 4 2*3 = 6 2*4 = 8 2*5 = 10 2*6 = 12 2*7 = 14 2*8 = 16 2*9 = 18	3*1 = 3 3*2 = 6 3*3 = 9 3*4 = 12 3*5 = 15 3*6 = 18 3*7 = 21 3*8 = 24 3*9 = 27
4*1 = 4 4*2 = 8 4*3 = 12 4*4 = 16 4*5 = 20 4*6 = 24 4*7 = 28 4*8 = 32 4*9 = 36	5*1 = 5 5*2 = 10 5*3 = 15 5*4 = 20 5*5 = 25 5*6 = 30 5*7 = 35 5*8 = 40 5*9 = 45	6*1 = 6 6*2 = 12 6*3 = 18 6*4 = 24 6*5 = 30 6*6 = 36 6*7 = 42 6*8 = 48 6*9 = 54
7*1 = 7 7*2 = 14 7*3 = 21 7*4 = 28 7*5 = 35 7*6 = 42 7*7 = 49 7*8 = 56 7*9 = 63	8*1 = 8 8*2 = 16 8*3 = 24 8*4 = 32 8*5 = 40 8*6 = 48 8*7 = 56 8*8 = 64 8*9 = 72	9*1 = 9 9*2 = 18 9*3 = 27 9*4 = 36 9*5 = 45 9*6 = 54 9*7 = 63 9*8 = 72 9*9 = 81

(最後留時間給同學們練習)

8.5 switch Multiple-Selection Statement

switch (條件變數){...動作...} //多選敘述

- switch multiple-selection statement
  - Tests a variable or expression separately for each value of cases and takes different actions accordingly
    - 註：逐一測試每個case值，並執行對應動作陳述
  - Consists of a series of case labels (多種情況) and an optional default (預設) case
    - 註：含多種case標籤，及一個"預設default"標籤 (可不加)
  - Compares the target value (目標值) with the value of each case labels (各情況值)
    - If it is true, the following statements are executed in order until a break statement is reached
    - 註：若目標值和case值相同，則執行接下來敘述直到break
- The default case (預設情況):
  - Usually the last case and used to specify the statements to execute if no other case is satisfied
    - 註：常作為最後的case，當不符合任何情況時執行。

(用法)

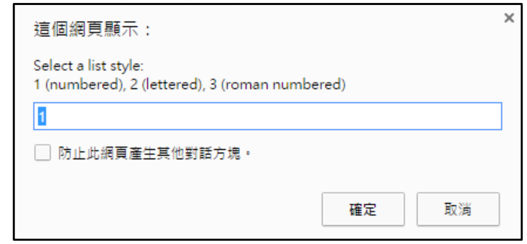
```
switch (...)
{
    case "...":
        ... //動作
        break;
    case "...":
        ... //動作
        break;
    default:
        ... //動作
        break;
}
```

8.5 switch Multiple-Selection Statement

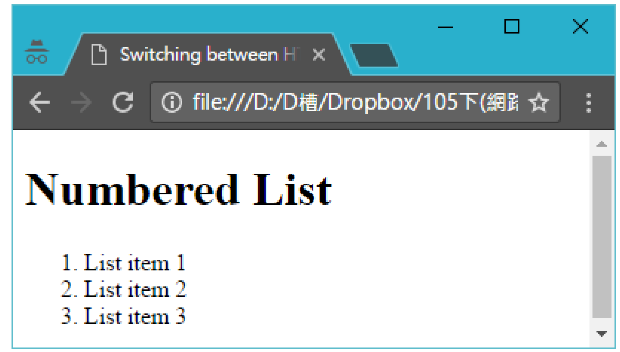
5) SwitchTest.html

```
<!DOCTYPE html>
<!-- Fig. 8.7: SwitchTest.html -->
<!-- Using the switch multiple-selection statement. -->
<html>
    <head>
        <meta charset = "utf-8">
        <title>Switching between HTML5 List Formats</title>
        <script>
            var choice; // user's choice
            var startTag; // starting list item tag
            var endTag; // ending list item tag
            var validInput = true; // true if input valid else false
            var listType; // type of list as a string
            choice = window.prompt( "Select a list style:\n" +
                "1 (numbered), 2 (lettered), 3 (roman numbered)", "1" );
            switch ( choice )                                //switch選擇
            {
                case "1":
                    startTag = "<ol>";
                    endTag = "</ol>";
                    listType = "<h1>Numbered List</h1>";
                    break;
                case "2":
                    startTag = "<ol style = 'list-style-type: upper-alpha'>";
                    endTag = "</ol>";
                    listType = "<h1>Lettered List</h1>";
                    break;
                case "3":
                    startTag = "<ol style = 'list-style-type: upper-roman'>";
                    endTag = "</ol>";
                    listType = "<h1>Roman Numbered List</h1>";
                    break;
                default: //其他 (不合法輸入：非輸入1、2、3)
                    validInput = false;
                    break;
            } //end switch
            if ( validInput === true )
            {
                document.writeln( listType + startTag );
                for ( var i = 1; i <= 3; ++i )
                    document.writeln( "<li>List item " + i + "</li>" );
                document.writeln( endTag );
            } //end if                                  顯示不同樣式
            else
                document.writeln( "Invalid choice: " + choice );
        </script>
    </head><body></body>
</html>
```

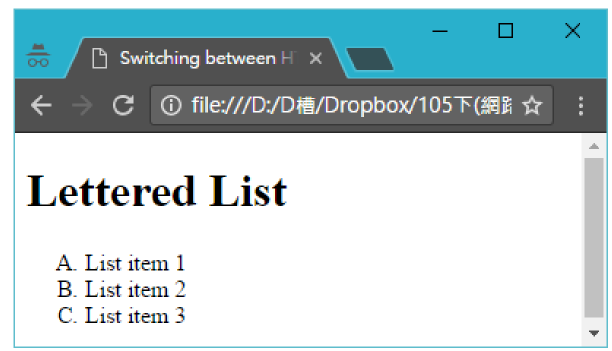
(輸入提醒)



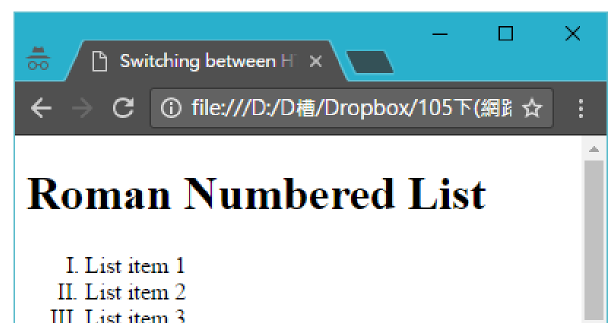
(case 1)



(case 2)



(case 3)



8.5 switch Multiple-Selection Statement

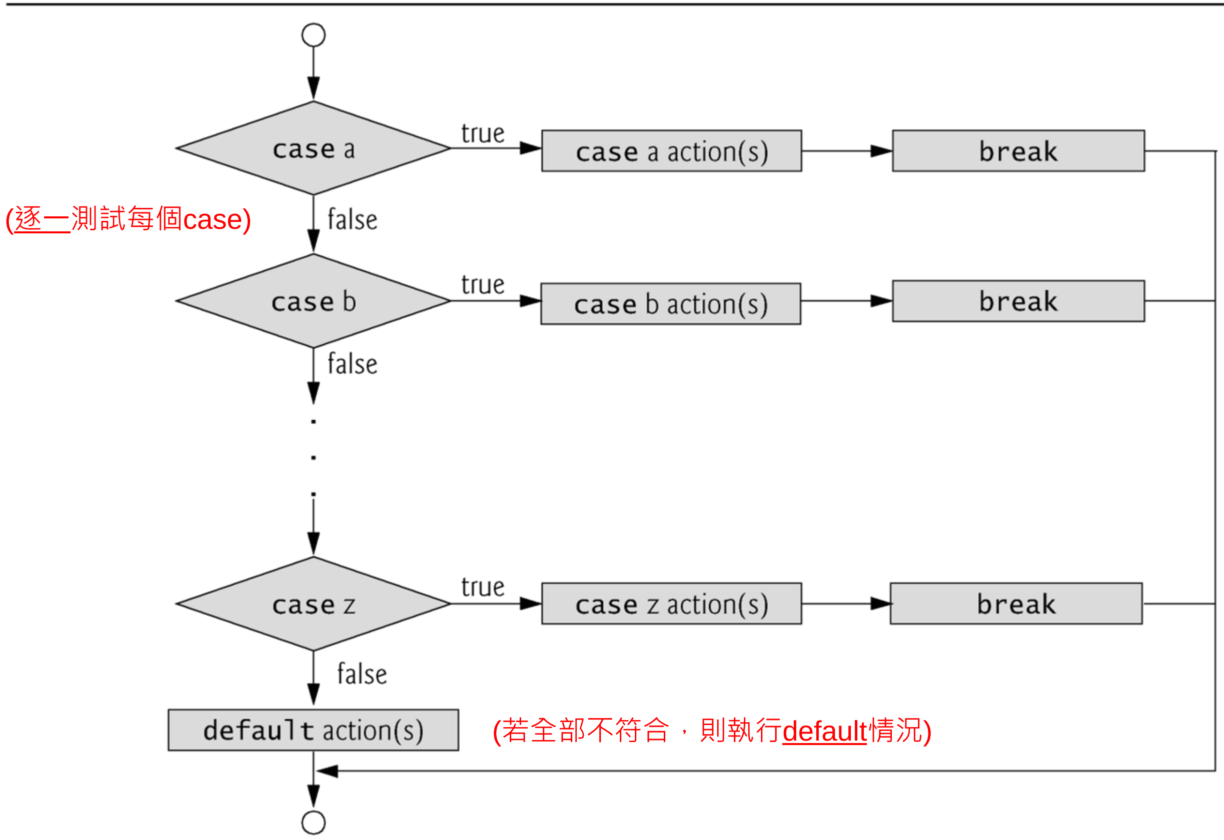


Fig. 8.8 | switch multiple-selection statement.

8.6 do...while Repetition Statement

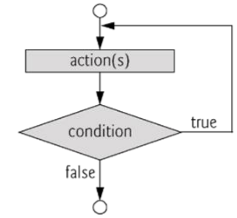


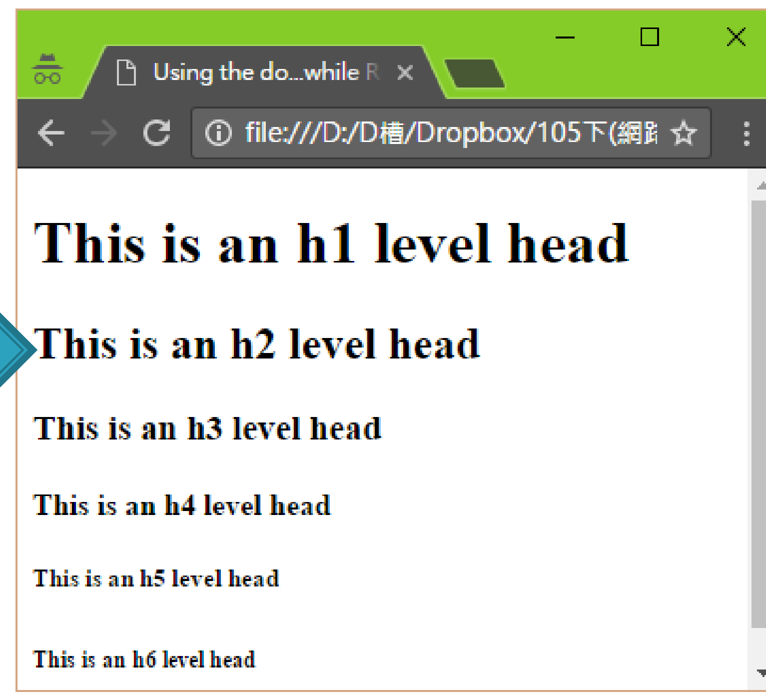
Fig. 8.10 | do...while repetition statement flowchart.

do{...動作...} while(...條件判斷...); //do-while迴圈

- do...while statement
  - Tests the loop condition *after* the loop body executes
    - 註：先執行，再測試條件
  - *The loop body always executes at least once (至少執行一次body內容)*

6) DoWhileTest.html

```html
<!DOCTYPE html>

<!-- Fig. 8.9: DoWhileTest.html -->
<!-- Using the do...while repetition statement. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>Using the do...while Repetition Statement</title>
      <script>

         var counter = 1;
                                        //do while迴圈
         do {
            document.writeln( "<h" + counter + ">This is " +
               "an h" + counter + " level head" + "</h" +
               counter + ">" );
            ++counter; //加1
         } while ( counter <= 6 );

      </script>

   </head><body></body>
</html>
```



8.7 break and continue Statements

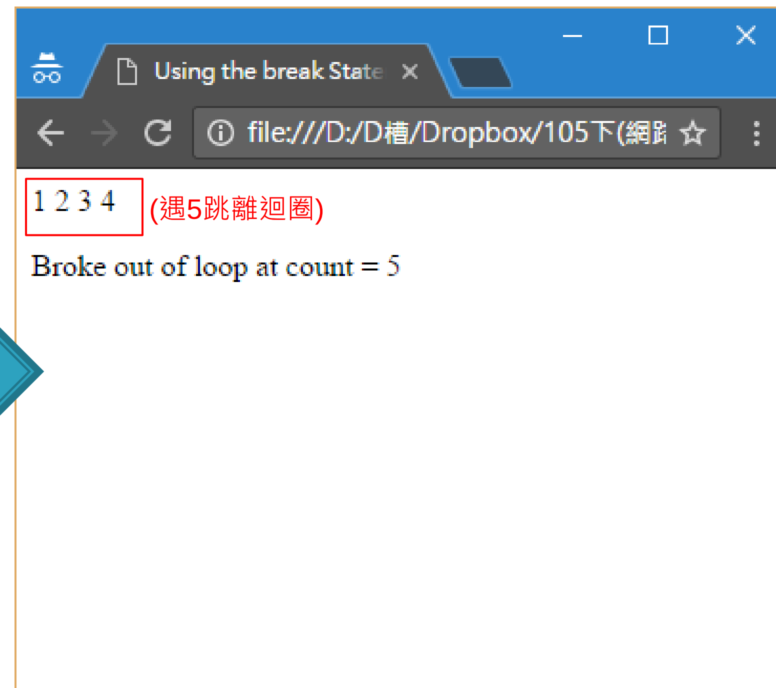
break; //中斷敘述

- break statement in a while, for, do…while or switch statement
  - Causes *immediate exit* (立即跳離) from the statement
  - 註：常用於跳出迴圈 (中斷迴圈)、結束switch case

7) BreakTest.html

```
<!DOCTYPE html>

<!-- Fig. 8.11: BreakTest.html -->
<!-- Using the break statement in a for statement. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>
         Using the break Statement in a for Statement
      </title>
      <script>

         for ( var count = 1; count <= 10; ++count )
         {
            if ( count == 5 )
               break;

            document.writeln( count + " " );
         } //end for

         document.writeln(
            "<p>Broke out of loop at count = " + count + "</p>" );

      </script>
   </head><body></body>
</html>
```

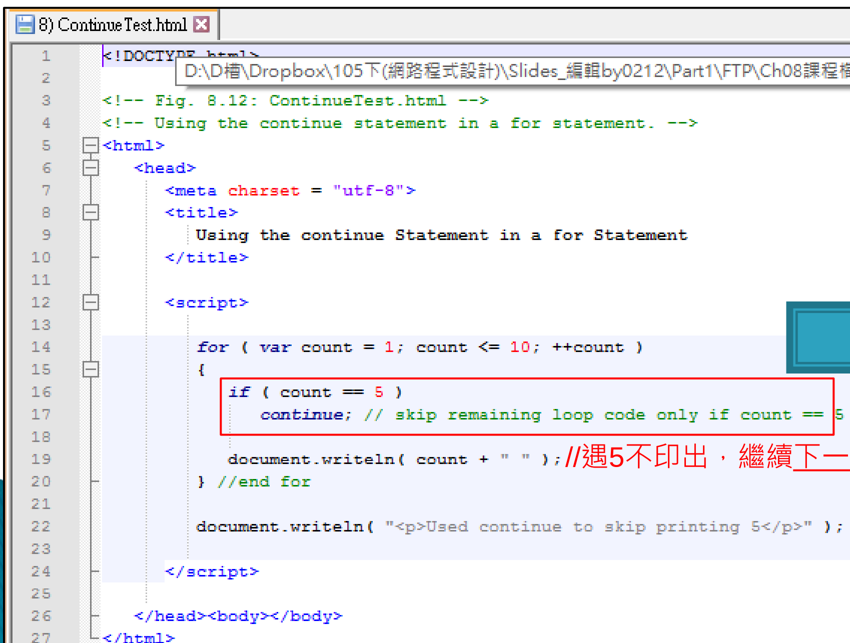
//遇到5，跳離for迴圈



8.7 break and continue Statements (Cont.)

continue; //直接跳到下一回合動作

- statement in a while, for or do…while
  - skips the remaining statements (跳離剩餘敘述) and proceeds with the next iteration of the loop (繼續執行下一輪敘述)
  - In while and do…while statements, the loop-continuation test evaluates immediately after the continue statement executes
    - 註：while及do while迴圈，會直接回到條件判斷式
  - In for statements, the increment expression executes, then the loop-continuation test evaluates
    - 註：(差異) for迴圈會「先執行遞增 / 遞減」，再進行條件判斷式




```
<!DOCTYPE html>

<!-- Fig. 8.12: ContinueTest.html -->
<!-- Using the continue statement in a for statement. -->
<html>
   <head>
      <meta charset = "utf-8">
      <title>
         Using the continue Statement in a for Statement
      </title>

      <script>

         for ( var count = 1; count <= 10; ++count )
         {
            if ( count == 5 )
               continue; // skip remaining loop code only if count == 5

            document.writeln( count + " " );
         } //end for

         document.writeln( "<p>Used continue to skip printing 5</p>" );

      </script>
   </head><body></body>
</html>
```

8.8 Logical Operators

邏輯敘述 (條件判斷)

- Logical operators can be used to form complex conditions (較複雜的條件) by combining simple conditions
  - && (logical AND: 兩者同時成立)
  - || (logical OR: 兩者擇一成立)
  - ! (logical NOT: 相反, also called logical negation)

… && … //同時成立   (ex:在13號 且 星期五，google網頁顯示不同)

- The && operator is used to ensure that two conditions are both true before choosing a certain path of execution
  - 註：確保2條件"同時成立"
  - 註：條件式可以為：relational operators (大於/小於), equality (等式).

expression1	expression2	expression1 && expression2
false	false	false
false	true	false
true	false	false
true	true	true

(2條件同時成立，結果才成立)

Fig. 8.1… | Truth table for the && (logical AND) operator.

8.8 Logical Operators (Cont.)

… || … //其一成立即可

- The || (logical OR) operator is used to ensure that either or both of two conditions are true
  - 註：2條件其中一者"成立"即可

expression1	expression2	expression1 || expression2
false	false	false
false	true	true
true	false	true
true	true	true

(2條件其中一者成立，即成立)

Fig. 8.14 | Truth table for the || (logical OR) operator.

優先順序

- The && operator has a higher precedence than the || operator
  - 註：&& 優先於 ||

結合性

(若沒有特別"括弧"的話)

- Both operators associate from left to right (由左至右).

8.8 Logical Operators (Cont.)

!(…) //邏輯反轉 (Ex: True→False, False→True)

- ! (logical negation) operator
  - Reverses (反轉) the meaning of a condition (i.e., a true value becomes false, and a false value becomes true)
  - Has only a single condition (單一條件運算) as an operand (i.e., it is a unary operator)
  - 註：放置在條件式的左方

expression		!expression	
false	→	true	(false變true)
true	→	false	(true變false)

Fig. 8.15 | Truth table for operator ! (logical negation).

8.9 Logical Operators (Cont.)

- 當使用數字判斷時：
  數字與布林數對應
  - Nonzero numeric values (非0數字) are considered to be true
    - 註：非0數皆視為True
  - The numeric value zero is considered to be false
    - 註：數字0皆視為False
- 當使用字串判斷時：
  字串與布林數對應
  - Any string that contains characters (非空字串) is considered to be true
    - 註：非空字串皆視為True
  - The empty string (空字串) is considered to be false
    - 註：空字串皆視為False
- 當使用物件判斷時：
  物件與布林數對應
  - All objects are considered to be true
    - 註：所有物件皆視為True
  - The value null and variables that have been declared but not initialized (未初始化) are considered to be False
    - 註：變數未初始 (或變數值為null)皆視為false

8.9 Logical Operators (Summary)

- 運算符號的結合性比較

Operator	Associativity	Type
++ -- !	right to left	unary (一元運算子，如：++a;)
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== != === !==	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment

Fig. 8.16 | Precedence and associativity of the operators discussed so far.

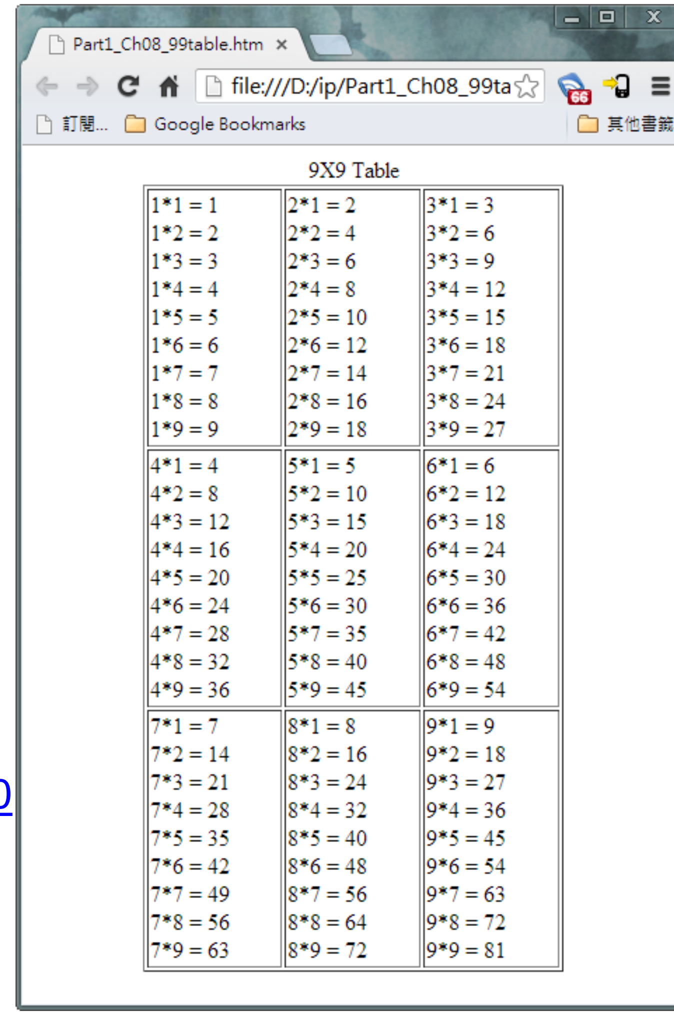
Lab
(上機練習)

In-Class Exercise #1

- Write a JavaScript program that reads in an arbitrary number of positive integers, and outputs the largest and the smallest integers.
- 課中練習：
  請使用JavaScript程式，讀取網頁使用者輸入的任意個正整數(請設定終止條件，如：-1)最後在網頁上顯示所輸入的1) 最大值、2) 最小值及3) 平均值。

In-Class Exercise #2

- Write a JavaScript program that produces the following 9X9 table.
- 課中練習：
  請用JavaScript建立如右圖的九九乘法表。



1*1 = 1 1*2 = 2 1*3 = 3 1*4 = 4 1*5 = 5 1*6 = 6 1*7 = 7 1*8 = 8 1*9 = 9	2*1 = 2 2*2 = 4 2*3 = 6 2*4 = 8 2*5 = 10 2*6 = 12 2*7 = 14 2*8 = 16 2*9 = 18	3*1 = 3 3*2 = 6 3*3 = 9 3*4 = 12 3*5 = 15 3*6 = 18 3*7 = 21 3*8 = 24 3*9 = 27
4*1 = 4 4*2 = 8 4*3 = 12 4*4 = 16 4*5 = 20 4*6 = 24 4*7 = 28 4*8 = 32 4*9 = 36	5*1 = 5 5*2 = 10 5*3 = 15 5*4 = 20 5*5 = 25 5*6 = 30 5*7 = 35 5*8 = 40 5*9 = 45	6*1 = 6 6*2 = 12 6*3 = 18 6*4 = 24 6*5 = 30 6*6 = 36 6*7 = 42 6*8 = 48 6*9 = 54
7*1 = 7 7*2 = 14 7*3 = 21 7*4 = 28 7*5 = 35 7*6 = 42 7*7 = 49 7*8 = 56 7*9 = 63	8*1 = 8 8*2 = 16 8*3 = 24 8*4 = 32 8*5 = 40 8*6 = 48 8*7 = 56 8*8 = 64 8*9 = 72	9*1 = 9 9*2 = 18 9*3 = 27 9*4 = 36 9*5 = 45 9*6 = 54 9*7 = 63 9*8 = 72 9*9 = 81

- 注意：
  - 請於23:59前上傳至Lab06：ftp://111webdesign:webdesign@120.114.190.22:4000
  - 請以學號建立資料匣(否則扣分)

【課程公告】

課程公告：(預計) 4/28 期中小考(上機考)

月份	週次	日	一	二	三	四	五	六
四月	七	5	6	7	8	9	10	11
	八	12	13	14	15	16	17	18
	九	19	20	21	22	23	24	25
	十	26	27	28	29	30		
五月							1	2
	十一	3	4	5	6	7	8	9
	十二	10	11	12	13	14	15	16
	十三	17	18	19	20	21	22	23
	十四	24	25	26	27	28	29	30

本週(上課：Ch0708)

繼續上課 (Ch09)

期中小考 (上機考)

期中-分組報告(1)

(備用) 期中-分組報告(2)

Ps. 期中小考(**上機考**)：將提供「課程講義」及「課程範例資料」下載

5/5, 5/12: 期中分組報告(Group Report)

- **說明**：進入期末專題前，讓各位同學具有分析網站優勢能力，期望能學習效法這些成功網站之元素，並提出小組期末專題構想 (網站/網頁應用)。

- 分組報告:
  - 1) 請上網搜尋一個成功知名的網站(頁)，如：
    - 社群類網站：Facebook、Instagram、Twitter、微博、抖音 (Tictok)、小紅書、Tumblr、Pinterest, Discord, Dcard, Plurk (噗浪)等 (不限於此)
    - 電子商務平台：Amazon、淘寶、蝦皮、ebay、Yahoo拍賣、露天、比價網 等 (不限於此)
    - 募資平台、旅遊/住宿網站、論壇網站、線上直播網、串流網站、影音平台等知名網站皆可
  - 2) 請分析介紹其成功原因，報告投影片大綱應包含：

    ★題目越獨特，會加分

    - 1. 簡介 (歷史、組成、客戶群…)
    - 2. 功能構成元素 (3~5項核心功能)
    - 3. 成功因素 (歸納出3~5項原因、持續成功原因、和其他平台差異、解決哪些痛點)
    - 4. 天馬行空的想像 & 進化版
    - 5. 你們的期末專題提案
    - 6. 參考資料
    - 簡報範例(連結)：https://drive.google.com/drive/folders/1QJFz8ln5BppWH_jkmWfWmHnIq0_dEqiO?usp=sharing

報告規則 & 評分標準

- 報告規則
  - 每組報告時間：15分鐘 (最多20分鐘，含Q&A)
  - 每組成員：1~3人一組 (最多3人)
  - 每組報告之題目(不可重覆)、順序(先報/後報)，請於期中考前4/27(一) 23:59 前mail至老師信箱: jmliang@mail.nutn.edu.tw (先寄先贏)
  - 報告順序，即時公佈在：
    - https://docs.google.com/spreadsheets/d/1lCve_2rLigQKjC85pGJUttPQ3w8PAHBbBA2GGFTQrJA/edit?usp=sharing
- 報告評分標準：
  - 主題獨特性(25%)
  - 內容完整性(25%)
  - 提案創意性(25%)
  - 投影片設計+表達清晰流暢(25%)

【工商服務】

【分享】:老師指導專題生-近期成果

- 【2025/10/23】
- 2025 NTIN-start 三創競賽
- 獲得「第一名 」
- https://www.instagram.com/nutn_ee/p/DQ

【分享】:老師指導專題生-前期成果

- 【2023/8/28】
- 第二十七屆行動計算研討會(Mobile Computing 2023)
- 獲得「優良論文獎 」

【分享】老師指導專題生-前期成果

- 【2022/10/27】
- 2022通訊大賽
- 獲得「校園菁英獎」
- 獎金：5萬元

- 【2022/11/06】
- 2022第27屆全國大專校院資訊應用創新競賽
- 獲得資訊應用組「第二名」
- 獎金：1萬元

【工商服務】❤按讚追蹤南大電機粉絲頁

- 國立臺南大學電機工程學系
- **IG粉絲頁**
  - https://www.instagram.com/nutn_ee/?hl=zh-tw

NUTN_EE

- 國立臺南大學電機工程學系
- **FB粉絲頁**
  - https://www.facebook.com/profile.php?id=100057285136331

南大電機